

Terraform Basic Notes

① What is Terraform?

- Infrastructure as Code (IaC) tool
- Define, provision, and manage cloud resources
- Uses declarative configuration

② Core Workflow

terraform init	→ initialize project
terraform plan	→ preview changes
terraform apply	→ create/update resources
terraform destroy	→ remove resources

③ Key Files

main.tf	resources and providers
variables.tf	input values
outputs.tf	useful results
terraform.tfstate	current infrastructure state

④ Basic Example

```
provider "aws" {  
  region = "eu-west-1"  
}  
  
resource "aws_s3_bucket" "demo" {  
  bucket = "my-demo-bucket"  
}
```

⑤ Important Ideas

- resource = thing Terraform manages
- provider = plugin for a platform
- variable = reusable input
- output = value shown after apply
- state = Terraform's source of truth

≧ Best practice: review plan before apply. Keep state secure. ≦

Terraform Workflow & HCL

① Terraform mental model

- Declarative: describe the desired end state
- Terraform compares config + state + real infrastructure
- It builds a dependency graph, then proposes changes

② Core workflow

terraform init	→ install providers, modules, backend
terraform fmt	→ format configuration
terraform validate	→ check syntax and consistency
terraform plan	→ preview changes
terraform apply	→ execute approved plan
terraform destroy	→ remove managed resources

③ HCL building blocks

provider	= platform plugin
resource	= object Terraform manages
data	= read existing information
variable	= input
local	= reusable expression
output	= published result

④ Basic HCL Example

```
terraform {  
  required_version = ">= 1.6.0"  
}  
  
provider "aws" {  
  region = var.region  
}  
  
resource "aws_s3_bucket" "logs" {  
  bucket = var.bucket_name  
}
```

≧ Always review the plan before apply. ≦

Terraform Commands Cheat Sheet

① Daily loop

- terraform fmt -recursive
- terraform init
- terraform validate
- terraform plan -out=tfplan
- terraform show tfplan
- terraform apply tfplan

② Inspection

- terraform providers
- terraform state list
- terraform state show ADDRESS
- terraform output -json
- terraform console
- terraform graph

③ Plan flags

- -var = set a variable
- -var-file = load variables from file
- -input=false = disable interactive prompts
- -auto-approve = skip confirmation (danger)
- -refresh-only = only refresh state

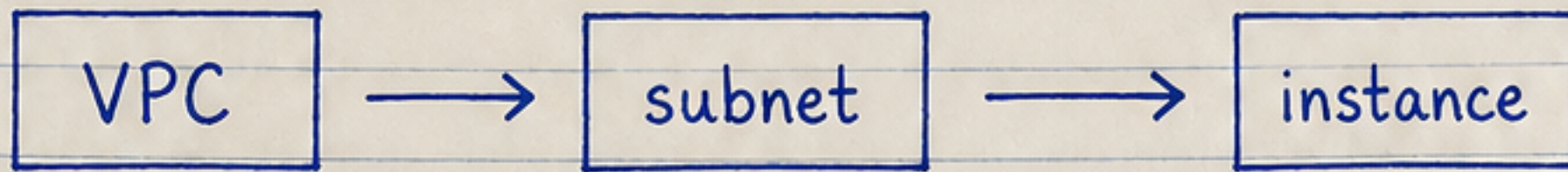
④ Never commit

.terraform/
terraform.tfstate
*.tfplan
secrets

➤ Save and review plans in CI/CD. ⚡

Expressions, Dependencies & Scale

① References create dependencies



② Meta-arguments

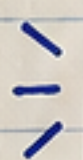
- count = create multiple similar resources
- for_each = iterate over a map or set
- depends_on = define explicit dependencies
- lifecycle = customize resource behavior
- provider = use a specific provider alias

③ for_each example

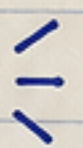
```
resource "aws_s3_bucket" "this" {  
  for_each = var.buckets  
  bucket   = each.value.name  
}
```

④ Useful functions

- merge = merge maps
- lookup = get value with default
- try = try multiple expressions
- coalesce = return first non-null value
- flatten = flatten nested lists
- toset = convert list to set
- jsonencode = encode to JSON string
- cidrsubnet = calculate subnet CIDR

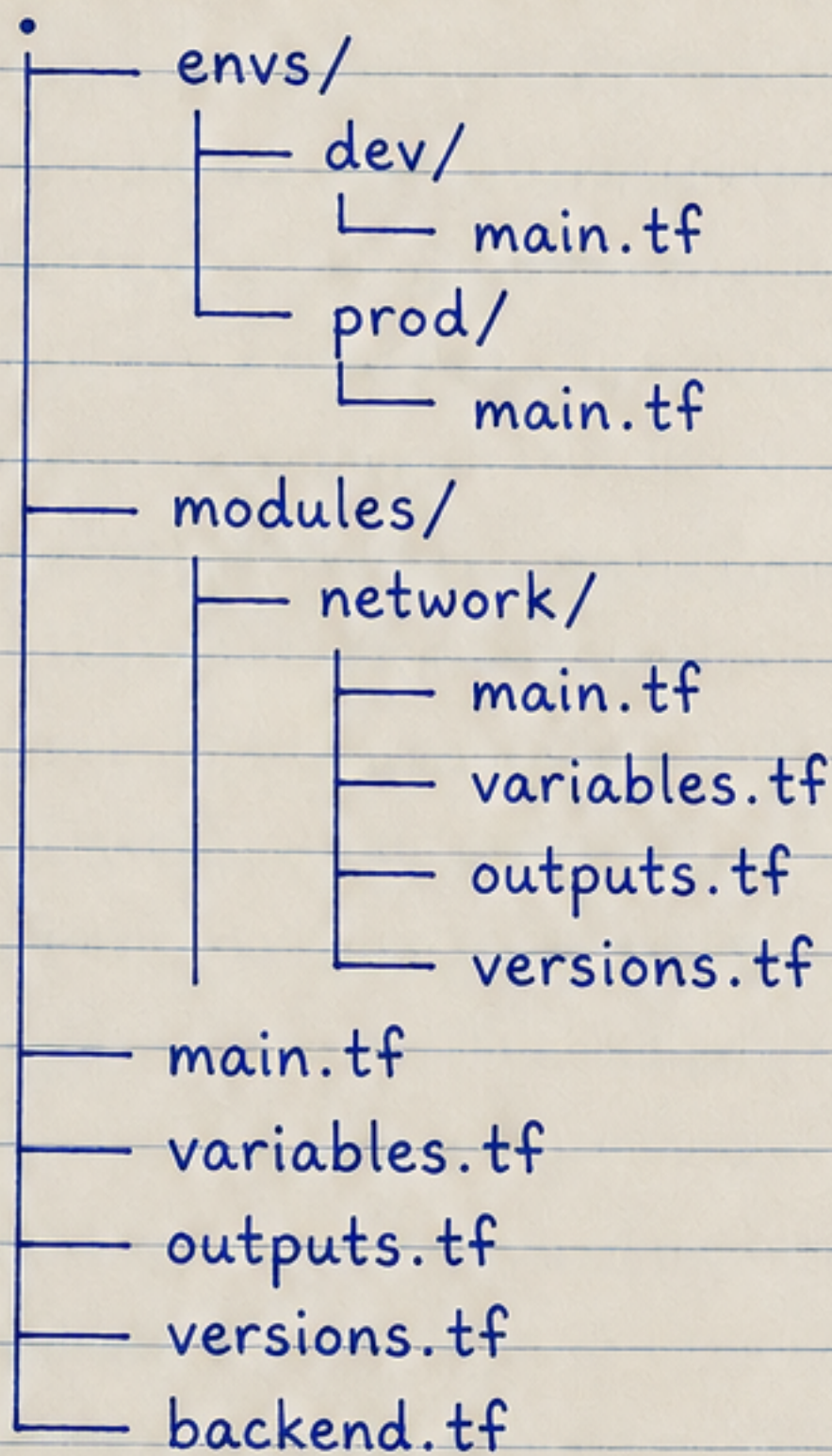


Rule: prefer explicit references over manual ordering.



Modules & Project Structure

① Project Structure



② What is a Module?

- module = reusable group of resources
- inputs go in
- outputs come out
- root module calls child modules

③ Example Module Call

```
module "network" {
  source = "../modules/network"
  name   = "dev"
  cidr   = "10.0.0.0/16"
}
```

④ Module Checklist

- pin versions
- document inputs/outputs
- validate inputs
- keep modules purposeful

Best practice: build small, composable modules.

State, Production & Troubleshooting

① State

- maps resource addresses to real IDs
- stores metadata
- keep it secure
- use remote backend + locking

② Advanced operations

- import blocks
- moved blocks
- lifecycle
- data sources
- workspaces

③ CI/CD flow

fmt → validate → plan → policy → approval → apply

④ Troubleshooting

- | | |
|------------------------|--|
| unexpected replacement | → first check: plan details / diff |
| drift | → first check: refresh + plan |
| lock error | → first check: who holds the lock |
| provider error | → first check: provider version & docs |

⑤ Important Ideas

- state = Terraform's source of truth
- production = safe, repeatable, observable
- troubleshoot = isolate, understand, fix
- logs + plan = best debugging tools
- remote state + locking = team safety

≧ Production rule: small changes, reviewed plans, pinned versions, protected state. ≦